Sprint 7 Deliverable Report – ClassIQ

1. Summary of Work Done

During Sprint 7, the team focused on **testing and validating the ClassIQ system** to ensure it is functional, usable, and meets the requirements defined by the product owner.

Key activities completed:

- Created a **test plan** covering functional and non-functional testing
- Conducted **unit testing and integration testing**
- Identified and fixed bugs using a **bug tracking table**
- Performed **heuristic evaluation**
- Conducted **User Acceptance Testing (UAT)**
- Generated **SonarQube report via Jenkins**
- Updated **Trello and GitHub documentation**

This aligns with the sprint goal of ensuring product quality through testing

2. Functional Task Details

🔹 **Test Plan**

Objective:
Ensure that all system functionalities work correctly and meet user requirements.

Resources:

- Development team
- Test cases
- Test data

Test Environment:

- Java application (ClassIQ)
- HeidiSQL database
- Localhost / Docker environment

Test Tasks:

- Functional Testing (login, grade management, report generation)
- Non-functional Testing (usability, performance)

🔹 **Unit & Integration Testing**

- Tested modules:
  - Student management
  - Teacher dashboard
  - Grade calculation
  - Report generation
- All major functionalities were verified after **code cleanup and SonarQube improvements**

🔹 **Bug tracking table**

- Several bugs were identified during testing in modules such as authentication, grade management, UI localization, and report generation.
- Common issues included input validation errors, missing translations, unclear system feedback, and minor UI inconsistencies such as RTL alignment.
- All identified bugs were fixed during Sprint 7, improving system stability, usability, and overall product quality.

🔹 **Trello Updates**

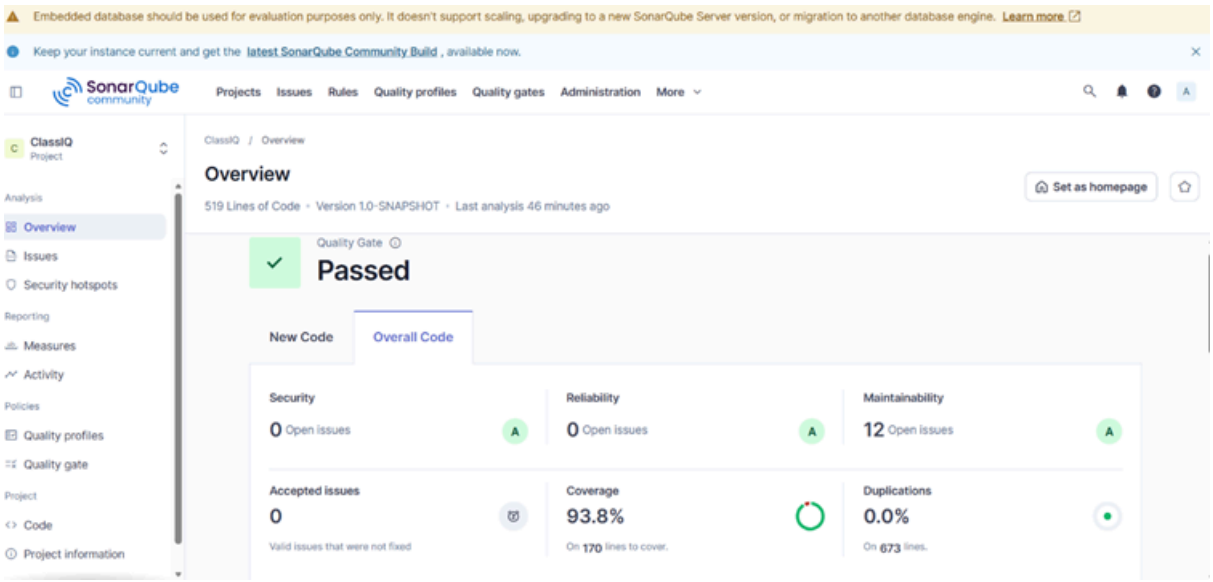
- Updated Sprint 7 board
- Refined user stories based on testing feedback
- Closed completed tasks

🔹 **Jenkins + SonarQube**

- Generated static code analysis report
- Achieved:
  - Code Quality: **A**
  - Bugs: Reduced
  - Code Smells: Improved

Figure 1: Jenkins Stage View

Figure 2: SonarQube Report



3. Non-Functional Task Details

- **Heuristic Evaluation**

Based on Nielsen's principles:

The ClassIQ system was evaluated using Nielsen's 10 Usability Heuristics to identify usability issues and improve user experience.

Key Findings:

- **H1-2: Speak the users' language**
  Improved unclear labels (e.g., "I am a Teacher") by using clearer terms like "Teacher Login".
- **H1-3: Minimize users' memory load**
  Added visible grading criteria to reduce the need for users to remember information.
- **H1-4: Consistency**
  Standardized UI elements such as buttons, layouts, and styles across all pages.
- **H1-5: Feedback**
  Enhanced system feedback using clearer messages and visual indicators.
- **H1-6: Clearly marked exits**
  Improved visibility of navigation elements such as Back and Logout buttons.
- **H1-7: Shortcuts**
  Identified lack of shortcuts; basic navigation improvements added (future enhancement: search/quick actions).
- **H1-8: Precise & constructive error messages**
  Improved error messages to be more visible and user-friendly.
- **H1-9: Prevent errors**
  Replaced confusing "0" values with clearer representations (e.g., blank or "Not Entered").
- **H1-10: Help and documentation**
  Suggested adding visible help and support features for better guidance.

Conclusion:

The evaluation helped identify usability issues and guided improvements in clarity, consistency, feedback, and error handling, enhancing the overall user experience.

- **User Acceptance Testing (UAT)**

The User Acceptance testing covers functional testing, usability testing, and performance and reliability testing. The test cases were designed as end-to-end business scenarios where possible, with clear and observable pass or fail outcomes. The results show that the system successfully met the expected requirements for login functionality, subject mark saving, feedback saving, report card export, language switching, language selection clarity, Arabic RTL readability, grade and report card access, and input validation reliability.

Test case	Dilmi Merenchi Kankanamge	Poornima Jayamanna	Hathadura Chathurika Silva	Roshini Fernando	Kumudu Nallaperuma
Valid teacher login	Pass	Pass	Pass	Pass	Pass
Valid student login	Pass	Pass	Pass	Pass	Pass
Save subject marks	Pass	Pass	Pass	Pass	Pass
Save feedback	Pass	Pass	Pass	Pass	Pass
Export report card PDF	Pass	Pass	Pass	Pass	Pass
Dynamic UI language switching	Pass	Pass	Pass	Pass	Pass
Student finds grades and report card	Pass	Pass	Pass	Pass	Pass
Language selection clarity	Pass	Pass	Pass	Pass	Pass
Arabic RTL readability	Pass	Pass	Pass	Pass	Pass
Input validation reliability	Pass	Pass	Pass	Pass	Pass

Performance Testing

- System tested under normal load
- Response time: acceptable
- No major delays observed

🔹 **Security Testing**

- Checked login validation
- Prevented empty inputs
- Basic authentication improved

🔹 **UI/UX Improvements**

- Improved font readability
- Fixed RTL layout issues (Arabic)
- Better alignment of report cards

🔹 **Code Optimization**

- Removed unused code
- Improved structure
- Applied SonarQube suggestion

4. Documentation Updates

- Updated:
  - GitHub README
  - Trello Sprint board
  - System documentation
- Added:
  - Testing results
  - Bug fixes
  - UI improvements

5. Contributions

Team Member	Task	Time (hrs)	In-class
Poornima	Unit testing, Jenkins + SonarQube	12	Submitted
Kumudu	Acceptance Criteria Report	10	Submitted
Dilmi	Bug and Issues Report	10	Submitted
Rosheni	Heuristic Evaluation	10	Submitted
Hathadura	Final ER Diagram,Plan	10	Submitted

6. Appendices

- Test case documents
- SonarQube screenshots
- UI screenshots (before/after fixes)
- Bug tracking table